

plenora-data-tools — Firme dei kernel (v1)

Documento generato dal codice sorgente del workspace (`crates/plenora-core`, `crates/plenora-kernels-table`, `crates/plenora-kernels-geo`). Copre le **127 operazioni** del catalogo unificato: 62 tabellari (`table.*`) e 65 geografiche (`geo.*`).

Come si legge una firma

Un piano v4 è un DAG dichiarativo (`PlanV4`): gli **input di dati** di un nodo arrivano dagli archi (`in`: riferimenti agli input dichiarati del piano o ad altri nodi; per le operazioni binarie ordinate l'ordine è semantico — [`left`, `right`]), mentre la **configurazione** è nominata nel nodo (`config`, oggetto JSON; `null` od omessa equivale a `{}`). La config viene deserializzata in modo tipizzato contro la struct `serde` dell'operazione, fail-closed: tutte le struct hanno `deny_unknown_fields`, quindi un campo sconosciuto è un errore. I vincoli ulteriori (campi non vuoti, range, colonne esistenti, tipi ammessi) sono verificati in `validate` del kernel e/o nell'analisi di contratto (`analyze.rs`), che inferisce anche lo schema dell'arco in uscita.

Esempio di nodo piano v4:

```
{
  "schema_version": 4,
  "crs": "EPSG:32632",
  "inputs": ["sorgente_a"],
  "nodes": [
    {
      "id": "n1",
      "op": "table.filter",
      "in": ["sorgente_a"],
      "config": {"column": "importo", "operator": ">", "value": 0}
    }
  ],
  "output": "n1"
}
```

Per ogni kernel il blocco riporta: id canonico, metadati di catalogo (`crates/plenora-core/src/catalog.rs`: provenienza, arietà, `execution class`; per le geo anche requisito CRS, capability richieste, result shape), la tabella dei campi config (campo | tipo JSON | default | vincoli), i requisiti sull'input e l'output prodotto. Convenzioni: `obbligatorio` = campo senza default `serde`; `Option<T>` senza default esplicito è riportato con default `null`. Il modulo `spill.rs` del crate tabellare non corrisponde ad alcuna operazione del catalogo (spilling interno degli operatori blocking) e non ha una firma.

Indice

Operazioni tabellari (62)

- **filtering** (2): `table.filter`, `table.conditional`
- **aggregation** (6): `table.sort`, `table.distinct`, `table.dedup_advanced`, `table.aggregate`, `table.rolling_window`, `table.window_function`
- **joins** (6): `table.join`, `table.semi_join`, `table.anti_join`, `table.asof_join`, `table.cross_join`, `table.concat`
- **cleansing** (3): `table.fill_na`, `table.replace`, `table.type_cast`
- **strings** (4): `table.string_pad`, `table.string_length`, `table.string_extract`, `table.text_normalize`
- **dates** (4): `table.date_format`, `table.date_add`, `table.date_diff`, `table.timezone_convert`
- **columns** (5): `table.drop_columns`, `table.rename`, `table.reorder_columns`, `table.concat_columns`, `table.split_column`
- **analysis** (5): `table.lookup`, `table.bin`, `table.flatten_json`, `table.statistics`, `table.sample`
- **reshape** (6): `table.melt`, `table.pivot`, `table.transpose`, `table.explode`, `table.unnest`, `table.table_diff`
- **setops** (3): `table.except`, `table.intersect`, `table.union_distinct`
- **security** (3): `table.md5_hash`, `table.sha256_hash`, `table.mask_data`
- **quality** (6): `table.assert_schema`, `table.assert_not_null`, `table.assert_unique`, `table.assert_range`, `table.assert_regex`, `table.coalesce`
- **governance** (4): `table.assert_cardinality`, `table.assert_metadata`, `table.assert_foreign_key`, `table.reconcile`
- **utility** (3): `table.add_row_number`, `table.date_extract`, `table.uuid_generator`
- **formula** (1): `table.formula`
- **expressions** (1): `table.expression`

Operazioni geografiche (65)

- **Geo Manipola-compat** (33): `geo.centroid`, `geo.convex_hull`, `geo.envelope`, `geo.sjoin`, `geo.area`, `geo.boundary`, `geo.bounds_extractor`, `geo.buffer`, `geo.clean_topology`, `geo.clip`, `geo.count_points_in_polygons`, `geo.difference`, `geo.dissolve`, `geo.distance`, `geo.explode`, `geo.from_coords`, `geo.intersection`, `geo.length`, `geo.line_builder`, `geo.nearest`, `geo.overlay`, `geo.perimeter`, `geo.point_on_surface`, `geo.polygon_builder`, `geo.simplify`, `geo.symmetric_difference`, `geo.to_wkt`, `geo.union`, `geo.vertex_count`, `geo.voronoi`, `geo.within`, `geo.make_valid`, `geo.reproject`
- **Predicati DE-9IM (estensioni geo)** (11): `geo.predicate_intersects`, `geo.predicate_disjoint`, `geo.predicate_contains`, `geo.predicate_within`, `geo.predicate_equals_topo`, `geo.predicate_covers`, `geo.predicate_covered_by`, `geo.predicate_contains_properly`, `geo.predicate_touches`, `geo.predicate_crosses`, `geo.predicate_overlaps`
- **Estensioni geo** (21): `geo.affine_transform`, `geo.translate`, `geo.scale`, `geo.rotate`, `geo.concave_hull`, `geo.hausdorff_distance`, `geo.haversine_distance`, `geo.geodesic_distance`, `geo.geodesic_line_length`, `geo.densify`, `geo.snap_to_grid`, `geo.delaunay`, `geo.polygonize`, `geo.line_merge`, `geo.split`, `geo.line_substring`, `geo.line_interpolate_point`, `geo.frechet_distance`, `geo.bearing`, `geo.geodesic_area`, `geo.geometry_diagnostics`

Tabellari — filtering

table.filter

manipola-compat · arietà: unaria · execution class: streaming · kernel v2

Config — Filter (filtering.rs:41)

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	colonna esistente in input
operator	string	obbligatorio	una di: ==, !=, >, >=, <, <=, contains, startswith, endswith, isnull, notnull, between
value	any (JSON)	null	per >/>=/</<= deve essere numerico; per between stringa "min,max" con estremi numerici; per ==/!= su colonna Int64/Float64 deve essere numerico

Input: una tabella; la colonna `column` deve esistere (qualsiasi tipo scalare). **Output:** stessa tabella con le righe che soddisfano il predicato; schema invariato (nessuna colonna aggiunta/rimossa), ordine relativo delle righe preservato.

table.conditional

manipola-compat · arietà: unaria · execution class: streaming

Config — Conditional (filtering.rs:65), con Condition (filtering.rs:50) e Operator (filtering.rs:18)

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	colonna esistente in input
conditions	array[object]	obbligatorio	elementi Condition (vedi sotto); operator/value soggetti agli stessi vincoli di table.filter
conditions[].operator	string	"=="	stesse varianti di Operator
conditions[].value	any (JSON)	null	come value di table.filter
conditions[].result	any (JSON)	null	letterale prodotto se la condizione è vera
default_value	any (JSON)	null	valore se nessuna condizione è vera
output_column	string	"result"	nome non vuoto, ≤ 1024 byte

Input: una tabella; la colonna `column` deve esistere. **Output:** aggiunge (o sostituisce se già presente) la colonna `output_column`; tipo Float64 nullable se tutti i letterali `result` e `default_value` sono vuoti o numerici, altrimenti Utf8 non nullable; numero righe invariato.

Tabellari — aggregation

table.sort

manipola-compat · arietà: unaria · execution class: blocking · kernel v2

Config — Sort (aggregation.rs:63)

Campo	Tipo	Default	Vincoli
columns	array[string]	obbligatorio	non vuoto; ogni colonna deve esistere nell'input
ascending	boolean	true	—

Input: tabella unaria; le colonne in `columns` devono esistere (tipi scalari: confronto nativo su Int64/Float64/Utf8/Boolean, percorso generico sugli altri tipi supportati). **Output:** stesse colonne dell'input; righe riordinate (sort stabile, null in coda in ascendente). Tabella.

table.distinct

manipola-compat · arietà: unaria · execution class: blocking

Config — Distinct (aggregation.rs:222)

Campo	Tipo	Default	Vincoli
subset	array[string]	[] (tutte le colonne)	ogni colonna elencata deve esistere ed essere scalare/stringa
keep	string (enum)	"first"	varianti: first, last, false

Input: tabella unaria; colonne in `subset` devono esistere (se vuoto, dedup su tutte le colonne). **Output:** stesse colonne dell'input; righe duplicate rimosse secondo `keep` (`first`: prima occorrenza, `last`: ultima, `false`: solo righe senza duplicati). Tabella.

table.dedup_advanced

manipola-compat · arietà: unaria · execution class: blocking

Config — DedupAdvanced (aggregation.rs:266)

Campo	Tipo	Default	Vincoli
subset	array[string]	obbligatorio	ogni colonna deve esistere ed essere scalare/stringa
keep	string (enum)	"first"	varianti: first, last; false non supportato (errore di contratto)
order_column	string	null	se presente, colonna esistente; sort interno ascendente prima della deduplica
ascending	boolean	true	direzione del sort interno su <code>order_column</code>

Input: tabella unaria; colonne in `subset` e l'eventuale `order_column` devono esistere. **Output:** stesse colonne dell'input; righe duplicate rimosse (come `table.distinct` con `keep first/last`), dopo eventuale ordinamento su `order_column`. Tabella.

table.aggregate

manipola-compat · arietà: unaria · execution class: blocking · kernel v2

Config — Aggregate (aggregation.rs:353) con elementi Aggregation (aggregation.rs:334) ed enum AggFunction (aggregation.rs:306)

Campo	Tipo	Default	Vincoli
group_by	array[string]	obbligatorio	non vuoto; ogni colonna deve esistere
aggregations	array[object]	[]	se vuoto viene prodotta solo la colonna count (Int64)
aggregations[].column	string	obbligatorio	colonna esistente; numerica per sum/avg/mean/min/max/variance/stddev/quantile, scalare/stringa per nunique/concat/first/last
aggregations[].function	string (enum)	"count"	varianti: count, sum, avg, mean, min, max, first, last, concat, nunique, variance, stddev, quantile
aggregations[].separator	string	", "	usato da concat
aggregations[].distinct	boolean	false	dedup dei valori prima dell'aggregazione (numeriche e concat)
aggregations[].skip_null	boolean	true	se false, un null nel gruppo rende nullo il risultato (numeriche) o conta come voce (nunique/concat)
aggregations[].alias	string	""	nome colonna output; se vuoto: column, oppure column_<funzione> se la stessa colonna appare in più aggregazioni
aggregations[].quantile	number	null	obbligatorio per function = "quantile"
aggregations[].ddof	integer	1	gradi di libertà per variance/stddev (gruppo con len <= ddof produce null)

Input: tabella unaria; colonne di `group_by` e di ogni aggregazione devono esistere, con tipo coerente con la funzione. **Output:** colonne di `group_by` (con tipo originario) più una colonna per aggregazione — `count/nunique` Int64 non nullable, `concat/first/last` Utf8 nullable, le altre Float64 nullable — oppure solo `count` (Int64) se aggregations è vuoto. Una riga per gruppo. Tabella.

table.rolling_window

estensione · arietà: unaria · execution class: blocking

Config — `RollingWindow` (`aggregation.rs:1106`) con enum `RollingKind` (`aggregation.rs:1092`)

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	colonna esistente, numerica
function	string (enum)	obbligatorio	varianti: sum, mean, min, max, stddev
group_by	string	null	se presente, colonna esistente; la finestra è calcolata per partizione
order_column	string	null	se presente, colonna esistente; sort interno ascendente prima del calcolo
window	integer	obbligatorio	> 0 e >= min_periods
min_periods	integer	1	> 0 e <= window
ddof	integer	1	per stddev (finestra con len <= ddof produce null)
output_column	string	obbligatorio	nome della colonna prodotta

Input: tabella unaria; `column` numerica, `group by/order_column` esistenti se specificati. **Output:** tutte le colonne dell'input più `output_column` (Float64 nullable), eventuale riordino su `order_column`. Stesse righe. Tabella.

table.window_function

manipola-compat · arietà: unaria · execution class: blocking

Config — `WindowFunction` (`aggregation.rs:1231`) con enum `WindowKind` (`aggregation.rs:1209`)

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	colonna esistente, numerica
function	string (enum)	"rank"	varianti: rank, dense_rank, cumsum, cumcount, lag, lead, pct_change, running_mean, percent_rank, cume_dist, ntile
group_by	string	null	se presente, colonna esistente; calcolo per partizione
order_column	string	null	se presente, colonna esistente; sort interno ascendente
offset	integer	1	> 0; usato da lag/lead
buckets	integer	null	ammesso solo con function = "ntile", dove è obbligatorio e > 0
output_column	string	null	nome colonna prodotta; se assente: <column>_<funzione>

Input: tabella unaria; `column` numerica, `group by/order_column` esistenti se specificati. **Output:** tutte le colonne dell'input più la colonna risultato (Float64 nullable; nome da `output_column` o <column>_<suffix>), eventuale riordino su `order_column`. Stesse righe. Tabella.

Tabellari — joins

table.join

manipola-compat · arietà: binaria (left, right) · execution class: binary-blocking

Config — `Join` (joins.rs:269)

Campo	Tipo	Default	Vincoli
left_keys	array[string]	obbligatorio	non vuoto; stessa cardinalità di right_keys; ogni nome deve esistere nell'input sinistro; tipi Arrow identici a coppie con right_keys
right_keys	array[string]	obbligatorio	ogni nome deve esistere nell'input destro; tipi Arrow identici a coppie con left_keys
how	string	"inner"	varianti: inner, left, right, outer (enum JoinHow, joins.rs:257, snake_case)

Input: due tabelle; colonne chiave presenti in entrambi gli input, con tipi Arrow identici a coppie (left/right).

Output: una tabella: tutte le colonne left (le non-chiave rinominate con suffisso `_L`, le chiavi col nome originale) seguite dalle colonne right non-chiave rinominate con suffisso `_R` (le chiavi right sono omesse; per `right/outer` il valore della chiave left è coalesciato con quello right); tutti i campi diventano nullable; le righe dipendono da `how` (match del join + righe non matchate secondo la variante); collisioni di nome residue → errore.

table.semi_join

estensione · arietà: binaria (left, right) · execution class: binary-blocking

Config — `MembershipJoin` (joins.rs:681)

Campo	Tipo	Default	Vincoli
left_keys	array[string]	obbligatorio	non vuoto; stessa cardinalità di right_keys; nomi esistenti nell'input sinistro; tipi Arrow identici a coppie
right_keys	array[string]	obbligatorio	nomi esistenti nell'input destro; tipi Arrow identici a coppie

Input: due tabelle; colonne chiave presenti in entrambi gli input, con tipi Arrow identici a coppie. **Output:** tabella con schema identico all'input sinistro (nessuna colonna aggiunta/rimossa/rinominata): solo le righe left la cui chiave matcha almeno una riga right, nell'ordine originale.

table.anti_join

estensione · arietà: binaria (left, right) · execution class: binary-blocking

Config — `MembershipJoin` (joins.rs:681)

Campo	Tipo	Default	Vincoli
left_keys	array[string]	obbligatorio	non vuoto; stessa cardinalità di right_keys; nomi esistenti nell'input sinistro; tipi Arrow identici a coppie
right_keys	array[string]	obbligatorio	nomi esistenti nell'input destro; tipi Arrow identici a coppie

Input: due tabelle; colonne chiave presenti in entrambi gli input, con tipi Arrow identici a coppie. **Output:** tabella con schema identico all'input sinistro (nessuna colonna aggiunta/rimossa/rinominata): solo le righe left la cui chiave NON matcha nessuna riga right, nell'ordine originale.

table.asof_join

estensione · arietà: binaria (left, right) · execution class: binary-blocking

Config — `AsOfJoin` (joins.rs:803)

Campo	Tipo	Default	Vincoli
left_on	string	obbligatorio	colonna esistente nell'input sinistro; tipo Int64 o Float64, identico a right_on
right_on	string	obbligatorio	colonna esistente nell'input destro; tipo Int64 o Float64, identico a left_on
left_by	array[string]	[]	stessa cardinalità di right_by; nomi esistenti; tipi Arrow identici a coppie con right_by
right_by	array[string]	[]	stessa cardinalità di left_by; nomi esistenti; tipi Arrow identici a coppie con left_by
direction	string	"backward"	varianti: backward, forward, nearest (enum AsOfDirection, joins.rs:791, snake_case)
tolerance	number	null	se presente: finita e ≥ 0
allow_exact	boolean	true	—

Input: due tabelle; colonne `left_on/right_on` numeriche (Int64/Float64) con tipo identico; eventuali colonne `by` presenti in entrambi con tipi identici a coppie. **Output:** una tabella con una riga per ogni riga left (nell'ordine left): tutte le colonne left invariate, poi le colonne right eccetto `right_on` e `right_by` (quelle che collidono con nomi left rinominate con suffisso `_R`); tutti i campi diventano nullable; le colonne right contengono null quando non

c'è match entro la tolleranza.

table.cross_join

manipola-compatible · *arietà: binaria (left, right)* · *execution class: binary-blocking*

Config — `CrossJoin (joins.rs:647)`

Config vuota (`{}`).

Input: due tabelle qualsiasi; nessun vincolo di chiave. **Output:** una tabella con righe = prodotto cartesiano delle cardinalità ($\text{left} \times \text{right}$, vincolato da `max_rows`): tutte le colonne left seguite da tutte le colonne right; i nomi presenti in entrambi gli input sono rinominati `_x` (left) e `_y` (right); tutti i campi diventano nullable; collisioni residue → errore.

table.concat

manipola-compatible · *arietà: n-aria* · *execution class: blocking*

Config — `Concat (joins.rs:585)`

Campo	Tipo	Default	Vincoli
ignore_index	boolean	true	—

Input: due o più tabelle (l'analyzer accetta N input) con schema identico: stesso numero di colonne, nomi e tipi Arrow identici campo per campo (nullability ignorata). **Output:** una tabella con lo schema del primo input (metadata inclusi; nullability = OR dei nullable degli input) e righe = somma delle righe degli input, nell'ordine dato.

Tabellari — cleansing

table.fill_na

manipola-compat · arietà: unaria · execution class: streaming · kernel v2

Config — FillNa (cleansing.rs:33)

Campo	Tipo	Default	Vincoli
column	string	null	opzionale: se assente, la fill si applica a tutte le colonne; se presente deve esistere
method	string	"value"	varianti: value, ffill, bfill
value	any (JSON)	null	con method=value, valore coerente col tipo della colonna (Int64: intero o stringa parsabile; Float64: numero o stringa con ./; Boolean: bool o "true"/"false"; Utf8: qualsiasi); null = nessuna sostituzione

Input: tabella; le colonne target devono essere di tipo Utf8, Int64, Float64 o Boolean (altri tipi, geometria inclusa, non sono supportati). **Output:** stesse colonne e stesso numero/ordine di righe; i valori null delle colonne target sostituiti (valore fisso, forward-fill o back-fill; i null in testa con ffill / in coda con bfill restano null).

table.replace

manipola-compat · arietà: unaria · execution class: streaming

Config — Replace (cleansing.rs:43)

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	deve esistere ed essere Utf8
old_value	string	obbligatorio	con regex=true deve essere una regex valida
new_value	string	obbligatorio	—
regex	boolean	false	—

Input: tabella con la colonna `column` di tipo Utf8. **Output:** stesse colonne (la colonna target riscritta in place come Utf8 nullable), stesse righe e ordine; sostituzione letterale o per regex.

table.type_cast

manipola-compat · arietà: unaria · execution class: streaming · kernel v2

Config — TypeCast (cleansing.rs:82)

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	deve esistere ed essere leggibile come scalare testuale (Utf8, Int64, Float64, Boolean, UInt64, Date32, Binary, ...)
target_type	string	"str"	varianti: str, int, float, bool, date, datetime, date32, timestamp_millis, decimal128, binary_utf8, uint64, dictionary_utf8
date_format	string	""	formato chrono per parsing date/datetime
errors	string	"coerce"	varianti: coerce, raise, ignore
precision	integer	null	obbligatorio con decimal128; 1..=38
scale	integer	null	obbligatorio con decimal128
timezone	string	null	solo con timestamp_millis; deve essere un nome IANA valido

Input: tabella con la colonna `column` di tipo scalare/testuale. **Output:** colonna target sostituita in place con il tipo Arrow corrispondente (str/date/datetime→Utf8, int→Int64, float→Float64, bool→Boolean, date32→Date32, timestamp_millis→Timestamp(ms[, tz]), decimal128→Decimal128(p,s), binary_utf8→Binary, uint64→UInt64, dictionary_utf8→Dictionary(Int32, Utf8)); i metadati di campo originali vanno persi (se era la colonna geometrica, il contratto diventa tabellare). Righe invariate.

Tabellari — strings

table.string_pad

manipola-compat · arietà: unaria · execution class: streaming

Config — `StringPad(strings.rs:17)`

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	deve esistere ed essere Utf8
width	integer	5	larghezza minima in caratteri Unicode
side	string	"left"	varianti: left, right
fill_char	string	"0"	esattamente un carattere Unicode
output_column	string	null	se assente sovrascrive column; nome valido se presente

Input: tabella con colonna Utf8. **Output:** colonna `output_column` (o `column` in place) di tipo Utf8 nullable con padding fino a `width`; righe invariate; errore se il risultato supera `max_string_bytes`.

table.string_length

manipola-compat · arietà: unaria · execution class: streaming

Config — `StringLength(strings.rs:104)`

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	deve esistere ed essere Utf8
output_column	string	null	default <column>_length; nome valido

Input: tabella con colonna Utf8. **Output:** aggiunge (o sostituisce) la colonna `output_column` di tipo Int64 nullable con il conteggio dei caratteri Unicode; righe invariate.

table.string_extract

manipola-compat · arietà: unaria · execution class: streaming

Config — `StringExtract(strings.rs:138)`

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	deve esistere ed essere Utf8
pattern	string	obbligatorio	regex valida; lunghezza ≤ <code>max_regex_bytes</code>
output_column	string	null	default <column>_extracted; ignorato se il pattern ha gruppi con nome
extract_all	boolean	false	se true concatena tutti i match separati da ,

Input: tabella con colonna Utf8. **Output:** se il pattern ha gruppi con nome, una colonna Utf8 nullable per gruppo (nome = nome del gruppo); altrimenti una colonna Utf8 nullable `output_column` con il primo gruppo di cattura (o l'intero match se non ci sono gruppi); null su input null o nessun match. Righe invariate.

table.text_normalize

manipola-compat · arietà: unaria · execution class: streaming · kernel v2

Config — `TextNormalize(strings.rs:293)`

Campo	Tipo	Default	Vincoli
columns	array[string]	obbligatorio	non vuoto; ogni colonna deve esistere ed essere Utf8
operations	string	"full"	varianti: trim, lower, upper, title, strip_accents, strip_double_spaces, full
overwrite	boolean	true	se false scrive in <name>_norm

Input: tabella con le colonne elencate di tipo Utf8. **Output:** per ogni colonna, la colonna normalizzata Utf8 nullable (in place con `overwrite=true`, altrimenti <name>_norm); righe invariate.

Tabellari — dates

table.date_format

estensione · arietà: unaria · execution class: streaming · kernel v2

Config — DateFormat (dates.rs:90)

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	colonna esistente, di tipo scalare leggibile come testo (Utf8, Int64, UInt64, Float64, Boolean, Date32, Timestamp, Binary)
input_format	string	obbligatorio	formato strftime chrono; item non riconosciuti fanno fallire il parsing di riga (gestito da invalid)
output_format	string	"%Y-%m-%d %H:%M:%S"	formato strftime chrono
output_column	string	obbligatorio	nome non vuoto, ≤ 1024 byte
invalid	string	"null"	varianti: null, error (InvalidDatePolicy); null → valore null in output, error → fallisce l'op

Input: una colonna (column) di tipo scalare leggibile come testo; i valori sono parsati come NaiveDateTime con fallback NaiveDate (orario 00:00:00). Righe invariate. **Output:** colonna output_column di tipo Utf8 (nullable) con il valore riformattato; sostituisce in posizione una colonna omonima esistente, altrimenti è aggiunta in coda.

table.date_add

estensione · arietà: unaria · execution class: streaming · kernel v2

Config — DateAdd (dates.rs:159)

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	colonna esistente, di tipo scalare leggibile come testo
input_format	string	obbligatorio	formato strftime chrono
output_format	string	"%Y-%m-%d %H:%M:%S"	formato strftime chrono
amount	integer	obbligatorio	quantità con segno; overflow del delta → valore null/errore secondo invalid
unit	string	obbligatorio	varianti: years, months, weeks, days, hours, minutes, seconds (DateUnit); anni/mesi con aritmetica calendariale (Months)
output_column	string	obbligatorio	nome non vuoto, ≤ 1024 byte
invalid	string	"null"	varianti: null, error

Input: una colonna (column) di tipo scalare leggibile come testo, parsata con input_format (fallback date-only). Righe invariate. **Output:** colonna output_column di tipo Utf8 (nullable) con la data traslata riformattata; upsert (sostituisce una colonna omonima, altrimenti appende).

table.date_diff

estensione · arietà: unaria · execution class: streaming · kernel v2

Config — DateDiff (dates.rs:268)

Campo	Tipo	Default	Vincoli
start_column	string	obbligatorio	colonna esistente, di tipo scalare leggibile come testo
end_column	string	obbligatorio	colonna esistente, di tipo scalare leggibile come testo
input_format	string	obbligatorio	formato strftime chrono, applicato a entrambe le colonne
unit	string	obbligatorio	varianti: days, hours, minutes, seconds (DiffUnit); differenza frazionaria
output_column	string	obbligatorio	nome non vuoto, ≤ 1024 byte
invalid	string	"null"	varianti: null, error

Input: due colonne (start_column, end_column) di tipo scalare leggibile come testo, parsate con input_format. Righe invariate. **Output:** colonna output_column di tipo Float64 (nullable): end - start in unità frazionarie (può essere negativa); intervallo fuori scala i64-ns → errore. Upsert sul nome.

table.timezone_convert

estensione · arietà: unaria · execution class: streaming · kernel v2

Config — TimezoneConvert (dates.rs:370)

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	colonna esistente, di tipo scalare leggibile come testo
input_format	string	obbligatorio	formato strftime chrono
output_format	string	"%Y-%m-%d %H:%M:%S"	formato strftime chrono
source_timezone	string	obbligatorio	nome IANA valido per chrono_tz::Tz (validato in analisi statica)
target_timezone	string	obbligatorio	nome IANA valido per chrono_tz::Tz (validato in analisi statica)
output_column	string	obbligatorio	nome non vuoto, ≤ 1024 byte
invalid	string	"null"	varianti: null, error
ambiguous	string	"error"	varianti: error, null, earliest, latest (AmbiguousPolicy); gestisce le ore ambigue DST; le ore inesistenti producono null/errore secondo invalid

Input: una colonna (`column`) di tipo scalare leggibile come testo; i valori naive sono localizzati in `source_timezone` e convertiti in `target_timezone`. Righe invariate. **Output:** colonna `output_column` di tipo Utf8 (nullable) con il timestamp convertito riformattato; upsert sul nome.

Tabellari — columns

table.drop_columns

manipola-compat · arietà: unaria · execution class: streaming

Config — DropColumns (columns.rs:15)

Campo	Tipo	Default	Vincoli
columns	array[string]	obbligatorio	nomi di colonne da rimuovere; nomi inesistenti ignorati (no-op)

Input: una tabella qualsiasi. **Output:** tabella senza le colonne elencate; righe invariate; se viene rimossa una colonna geometria, i metadati geo associati vengono eliminati.

table.rename

manipola-compat · arietà: unaria · execution class: streaming

Config — Rename (columns.rs:48), con RenamePair (columns.rs:41)

Campo	Tipo	Default	Vincoli
renames	array[object]	obbligatorio	elementi RenamePair (vedi sotto)
renames[].old_name	string	obbligatorio	colonna inesistente ignorata; in caso di old_name duplicati vince l'ultima coppia
renames[].new_name	string	obbligatorio	nome non vuoto, ≤ 1024 byte; il risultato non deve produrre nomi duplicati

Input: una tabella qualsiasi. **Output:** stessa tabella con le colonne rinominate; righe, ordine e tipi invariati; i metadati geometria seguono il nuovo nome.

table.reorder_columns

manipola-compat · arietà: unaria · execution class: streaming

Config — ReorderColumns (columns.rs:82)

Campo	Tipo	Default	Vincoli
columns	array[string]	[]	nomi di colonne esistenti, senza duplicati
alphabetical	boolean	false	alias serde: sort_alphabetical

Input: una tabella qualsiasi. **Output:** stesse colonne e stesse righe, riordinate: prima le colonne in `columns` (nell'ordine dato), poi le restanti (in ordine alfabetico case-insensitive se `alphabetical` è true, altrimenti nell'ordine originale).

table.concat_columns

manipola-compat · arietà: unaria · execution class: streaming

Config — ConcatColumns (columns.rs:127)

Campo	Tipo	Default	Vincoli
columns	array[string]	obbligatorio	non vuoto; ogni colonna deve esistere ed essere Utf8
output_column	string	"concatenated"	nome non vuoto, ≤ 1024 byte
separator	string	" "	—
skip_null	boolean	true	—

Input: una tabella; tutte le colonne in `columns` devono essere di tipo Utf8. **Output:** aggiunge (o sostituisce) la colonna `output_column` di tipo Utf8 nullable; righe invariate. Con `skip_null` true i null sono omessi e la riga è null se tutti i valori sono null; con false i null contano come stringa vuota. Errore se il risultato supera `max_string_bytes`.

table.split_column

manipola-compat · arietà: unaria · execution class: streaming

Config — SplitColumn (columns.rs:198)

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	colonna esistente di tipo Utf8
delimiter	string	","	non vuoto
new_columns	array[string]	obbligatorio	non vuoto; nomi univoci, non vuoti, ≤ 1024 byte; numero ≤ <code>max_split_columns</code>
max_splits	integer	-1	-1 (o ≤ 0) = illimitato; se > 0 limita il numero di split

Input: una tabella; la colonna `column` deve essere Utf8. **Output:** aggiunge (o sostituisce) le colonne in

`new_columns`, tutte Utf8 nullable, con le parti dello split (null oltre le parti disponibili); la colonna sorgente resta; righe invariate.

Tabellari — analysis

table.lookup

manipola-compat · arietà: unaria · execution class: streaming

Config — Lookup (analysis.rs:19)

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	deve esistere; colonna stringa scalare
mapping	object[string -> any]	obbligatorio	chiavi = valori sorgente, valori JSON qualsiasi (stringhe usate verbatim, altri serializzati)
default	any (valore JSON)	null	null = mantiene il valore originale se assente da mapping
output_column	string	null	null = sovrascrive column in place; nome output valido

Input: 1 tabella; colonna `column` di tipo stringa scalare. **Output:** stessa tabella con colonna Utf8 `output_column` (o `column` sovrascritta); righe invariate.

table.bin

manipola-compat · arietà: unaria · execution class: blocking

Config — Bin (analysis.rs:80), enum Bins (analysis.rs:69, serde untagged)

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	deve esistere; colonna numerica
bins	integer oppure array[number]	5	intero = numero di bin equal-width, in 2..=100; array = bordi espliciti, lunghezza 3..=101, strettamente crescenti
labels	array[string]	null	se presente, lunghezza = numero di bin
output_column	string	null	null = <column>_bin

Input: 1 tabella; colonna `column` numerica. **Output:** aggiunge colonna Utf8 <column>_bin (o `output_column`) con label del bin ((a, b] o etichetta custom); righe invariate.

table.flatten_json

manipola-compat · arietà: unaria · execution class: streaming

Config — FlattenJson (analysis.rs:207)

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	deve esistere; colonna stringa scalare (contenuto JSON)
prefix	string	""	stringa vuota = prefisso effettivo <column>_; ogni output deve iniziare con il prefisso
max_level	integer	1	massimo 5
output_columns	array[string]	[]	se vuoto i nomi derivano dai dati (schema non inferibile a secco); ogni nome deve iniziare con prefix e essere un nome valido

Input: 1 tabella; colonna `column` stringa contenente oggetti JSON. **Output:** aggiunge una colonna Utf8 per ogni chiave appiattita (o per ogni nome in `output_columns`); righe invariate; vincolo `max_columns`.

table.statistics

manipola-compat · arietà: unaria · execution class: blocking

Config — Statistics (analysis.rs:335), enum Stat (analysis.rs:309, snake_case)

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	deve esistere; colonna numerica
group_by	string	null	se presente deve esistere
stats	array[string]	["count", "min", "max", "mean", "median", "std"]	varianti: count, min, max, sum, mean, median, std, var, q25, q75
output_prefix	string	""	stringa vuota = prefisso effettivo <column>_

Input: 1 tabella; colonna `column` numerica; opzionale colonna `group_by`. **Output:** aggiunge una colonna Float64 <prefix><stat> per ogni statistica richiesta, con valore del gruppo replicato (broadcast) su ogni riga; righe invariate.

table.sample*manipola-compatible · arietà: unaria · execution class: blocking***Config** — `Sample (analysis.rs:452)`

Campo	Tipo	Default	Vincoli
n	integer	100	usato se fraction assente; conteggio effettivo min(n, righe)
fraction	number	null	in 0.0..=1.0; se presente ha priorità su n
random_state	integer	null	seed dello shuffle; null = seed fisso interno
stratify_column	string	null	se presente deve esistere ed essere stringa scalare; campionamento per gruppo

Input: 1 tabella; opzionale colonna `stratify_column` stringa scalare. **Output:** stesso schema; sottoinsieme di righe (`min(n, righe)` oppure `round(righe * fraction)`; con `stratify` almeno 1 riga per gruppo); nessun ordinamento preservato.

Tabellari — reshape

table.melt

manipola-compat · arietà: unaria · execution class: blocking

Config — Melt (reshape.rs:22), enum HeterogeneousTypePolicy (reshape.rs:36, snake_case)

Campo	Tipo	Default	Vincoli
id_columns	array[string]	obbligatorio	ogni colonna deve esistere
value_columns	array[string]	[]	vuoto = tutte le colonne non id; almeno una colonna valore effettiva richiesta
var_name	string	"variable"	nome valido; in caso di collisione viene suffissato (_1, _2, ...)
value_name	string	"value"	come var_name
type_policy	string	"reject"	varianti: reject, string; reject richiede value columns omogenee per tipo Arrow

Input: 1 tabella; colonne `id_columns` e `value_columns` esistenti. **Output:** colonne `id_columns` + `var_name` (Utf8, nome colonna sorgente) + `value_name` (tipo comune, oppure Utf8 con `type_policy` = "string");
righe = `righe_input` × numero `value_columns`.

table.pivot

manipola-compat · arietà: unaria · execution class: blocking

Config — Pivot (reshape.rs:346), enum PivotAgg (reshape.rs:330, snake_case)

Campo	Tipo	Default	Vincoli
index_col	string	obbligatorio	una o più colonne separate da virgola; ognuna deve esistere
pivot_col	string	obbligatorio	deve esistere (nome JSON <code>pivot_col</code> , campo Rust <code>column</code>)
value_col	string	obbligatorio	deve esistere
aggr_func	string	"first"	varianti: first, last, max, min, sum, mean, count, concat
mapping	object[string -> string]	{}	filtra i valori pivot ammessi e rinomina le colonne di output; vuoto = tutti i valori, nomi grezzi

Input: 1 tabella; colonne `index_col`, `pivot_col`, `value_col` esistenti. **Output:** colonne indice + una colonna per ogni valore distinto di `pivot_col` (tipo: `first/last` = tipo di `value_col`; `count` = Int64; `concat` = Utf8; `sum/mean/min/max` = Float64); una riga per combinazione distinta delle colonne indice. Schema non inferibile a secco (dipende dai dati).

table.transpose

manipola-compat · arietà: unaria · execution class: blocking

Config — Transpose (reshape.rs:520)

Campo	Tipo	Default	Vincoli
id_column	string	null	se presente deve esistere; i suoi valori diventano i nomi delle colonne di output
output_columns	array[string]	[]	nomi espliciti per le colonne di output (per posizione di riga); sovrascrivono i default
type_policy	string	"reject"	varianti: reject, string; reject richiede colonne dati omogenee per tipo Arrow

Input: 1 tabella; opzionale colonna `id_column`. **Output:** prima colonna (nome = `id_column` oppure `col_0`, Utf8) con i nomi delle colonne dati + una colonna per riga dell'input (tipo comune, oppure Utf8 con `type_policy` = "string"); righe = numero colonne dati. Schema non inferibile a secco (dipende dal numero di righe).

table.explode

estensione · arietà: unaria · execution class: blocking

Config — Explode (reshape.rs:645), enum EmptyListPolicy (reshape.rs:634, snake_case)

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	deve esistere; colonna di tipo List
output_column	string	null	null = sostituisce <code>column</code> in place; nome output valido
empty_policy	string	"null"	varianti: drop, null; gestione di liste vuote o null

Input: 1 tabella; colonna `column` di tipo Arrow List. **Output:** colonna esplosa (`output_column` o `column`) con il tipo elemento della lista; una riga per elemento (le altre colonne replicate); con `empty_policy` = "null" le liste vuote/null producono una riga con valore null, con `drop` nessuna.

table.unnest

estensione · arietà: unaria · execution class: streaming

Config — Unnest (reshape.rs:698)

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	deve esistere; colonna di tipo Struct
prefix	string	""	prefisso dei nomi delle colonne figlie espanse
drop_source	boolean	true	true = rimuove la colonna struct dall'output

Input: 1 tabella; colonna `column` di tipo Arrow Struct. **Output:** colonne originali (meno la struct se `drop_source = true`) + una colonna per campo dello struct, nominata `<prefix><campo>` (nullable); righe invariate; errore su collisione di nomi; vincolo `max_columns`.

table.table_diff

manipola-compat · arietà: binaria (left, right) · execution class: binary-blocking

Config — TableDiff (reshape.rs:770)

Campo	Tipo	Default	Vincoli
left_keys	array[string]	obbligatorio	non vuoto; stessa cardinalità di <code>right_keys</code> ; colonne esistenti in left, tipi identici alle <code>right_keys</code>
right_keys	array[string]	obbligatorio	vedi <code>left_keys</code> ; chiavi duplicate in left o right = errore
compare_columns	array[string]	[]	vuoto = colonne di left non chiave presenti anche in right; ogni colonna deve esistere in entrambe con tipo Arrow identico
include_unchanged	string	"no"	"yes" = emette anche le righe UNCHANGED
separator	string	"#"	separatore delle liste in <code>_diff_columns</code> / <code>_diff_old_values</code>

Input: 2 tabelle (left, right); colonne chiave e di confronto come sopra. **Output:** colonne `left_keys` (nullable) + `compare_columns` (nullable, valori della riga più recente) + `_diff_status` (Utf8: ADDED, DELETED, MODIFIED, UNCHANGED) + `_diff_columns` e `_diff_old_values` (Utf8 nullable); una riga per chiave distinta (prima le chiavi di left, poi quelle solo di right).

Tabellari — setops

table.except

estensione · arietà: binaria (left, right) · execution class: binary-blocking

Config — `SetOperation (setops.rs:18)`

Config vuota (`{}`).

Input: due tabelle con schema identico (nomi e tipi Arrow identici campo per campo); ogni colonna deve avere un tipo supportato dall'encoder di riga: Utf8, Int64, UInt64, Float64, Boolean, Date32, Timestamp(ms), Decimal128, Binary, Dictionary(Int32, Utf8). **Output:** tabella con schema identico all'input sinistro: righe distinte di left che non compaiono in right (semantica EXCEPT DISTINCT), nell'ordine canonico di prima occorrenza.

table.intersect

estensione · arietà: binaria (left, right) · execution class: binary-blocking

Config — `SetOperation (setops.rs:18)`

Config vuota (`{}`).

Input: due tabelle con schema identico (nomi e tipi Arrow identici campo per campo); tipi colonna limitati come per `table.except`. **Output:** tabella con schema identico all'input sinistro: righe distinte di left che compaiono anche in right (semantica INTERSECT DISTINCT), nell'ordine canonico di prima occorrenza.

table.union_distinct

estensione · arietà: binaria (left, right) · execution class: binary-blocking

Config — `SetOperation (setops.rs:18)`

Config vuota (`{}`).

Input: due tabelle con schema identico (nomi e tipi Arrow identici campo per campo); tipi colonna limitati come per `table.except`. **Output:** una tabella con lo schema dell'input sinistro (`nullability = OR` dei due input) e righe = concatenazione di left e right con duplicati di riga intera rimossi (UNION DISTINCT), in ordine di prima occorrenza.

Tabellari — security

table.md5_hash

manipola-compat · arietà: unaria · execution class: streaming

Config — Md5Hash (security.rs:14)

Campo	Tipo	Default	Vincoli
columns	array[string]	obbligatorio	non vuoto; colonne esistenti di tipo scalare testuale (Utf8, Int64, UInt64, Float64, Boolean, Date32, Binary, Timestamp(ms), Decimal128, Dictionary(Int32,Utf8))
output_column	string	"md5_hash"	nome colonna valido
normalize	boolean	true	trim + lowercase prima dell'hash
null_policy	string	"empty"	enum HashNullPolicy (snake_case): empty, literal, error
null_literal	string	"<null>"	usato solo con null_policy = "literal"

Input: tabella; colonne elencate in `columns` (tipo scalare testuale). **Output:** stesse righe; colonna `output_column` Utf8 non-null aggiunta in coda (o sovrascritta se il nome esiste già); digest MD5 esadecimale per riga.

table.sha256_hash

estensione · arietà: unaria · execution class: streaming

Config — Sha256Hash (security.rs:100)

Campo	Tipo	Default	Vincoli
columns	array[string]	obbligatorio	può essere vuoto (a differenza di md5_hash); colonne esistenti di tipo scalare testuale
output_column	string	"sha256_hash"	nome colonna valido
normalize	boolean	true	trim + lowercase prima dell'hash
null_policy	string	"empty"	enum HashNullPolicy (snake_case): empty, literal, error
null_literal	string	"<null>"	usato solo con null_policy = "literal"

Input: tabella; colonne elencate in `columns` (tipo scalare testuale). **Output:** stesse righe; colonna `output_column` Utf8 non-null aggiunta/sovrascritta; digest SHA-256 esadecimale con framing per nome colonna e tipo.

table.mask_data

manipola-compat · arietà: unaria · execution class: streaming

Config — MaskData (security.rs:221), con sotto-config Masking (security.rs:207)

Campo	Tipo	Default	Vincoli
maskings	array[Masking]	obbligatorio	non vuoto; applicate in sequenza (una masking può riferirsi a una colonna <code>_masked</code> creata da una precedente)
overwrite	boolean	false	se true sovrascrive la colonna originale, altrimenti crea <code><colonna>_masked</code>

Campi di Masking:

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	colonna esistente, tipo scalare testuale
mask_type	string	"custom"	enum MaskType (snake_case): cf, email, phone, iban, custom
chars_start	integer	3	usato solo con mask_type = "custom"
chars_end	integer	3	usato solo con mask_type = "custom"
mask_char	string	""	un solo carattere (verificato solo per custom)

Input: tabella; colonne indicate nelle masking (tipo scalare testuale). **Output:** stesse righe; per ogni masking una colonna Utf8 nullable: `<colonna>_masked` aggiunta, oppure la colonna originale sostituita se `overwrite = true`.

Tabellari — quality

table.assert_schema

estensione · arietà: unaria · execution class: streaming

Config — AssertSchema (quality.rs:24), con sotto-config SchemaExpectation (quality.rs:16)

Campo	Tipo	Default	Vincoli
fields	array[SchemaExpectation]	obbligatorio	—
allow_extra	boolean	false	se false il numero di colonne deve coincidere con fields.len()
ordered	boolean	true	se true il confronto è posizionale, altrimenti per nome

Campi di SchemaExpectation:

Campo	Tipo	Default	Vincoli
name	string	obbligatorio	—
data_type	string	obbligatorio	uno tra: utf8/string, int64/integer, float64/float/double, boolean/bool, uint64/unsigned, date32, timestamp_millis, decimal128, binary, dictionary_utf8, list, struct
nullable	boolean	null	se presente, la nullability della colonna deve coincidere

Input: tabella qualsiasi. **Output:** input invariato; errore se lo schema non è conforme.

table.assert_not_null

estensione · arietà: unaria · execution class: streaming

Config — AssertNotNull (quality.rs:132)

Campo	Tipo	Default	Vincoli
columns	array[string]	obbligatorio	colonne esistenti (qualsiasi tipo)

Input: tabella; colonne elencate in columns. **Output:** input invariato; errore al primo valore null trovato.

table.assert_unique

estensione · arietà: unaria · execution class: blocking

Config — AssertUnique (quality.rs:171)

Campo	Tipo	Default	Vincoli
columns	array[string]	obbligatorio	colonne esistenti di tipo scalare testuale
nulls_equal	boolean	true	se false le righe con almeno un null sono escluse dal controllo

Input: tabella; colonne elencate in columns (tipo scalare testuale). **Output:** input invariato; errore alla prima combinazione di chiavi duplicata.

table.assert_range

estensione · arietà: unaria · execution class: streaming

Config — AssertRange (quality.rs:203)

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	colonna esistente di tipo numerico (Float64, Int64, UInt64, Date32, Timestamp(ms), Decimal128, Utf8)
min	number	null	—
max	number	null	—
inclusive_min	boolean	true	—
inclusive_max	boolean	true	—
allow_null	boolean	false	se false un null è errore

Input: tabella; colonna numerica indicata. **Output:** input invariato; errore per valori non finiti o fuori intervallo.

table.assert_regex

estensione · arietà: unaria · execution class: streaming

Config — AssertRegex (quality.rs:252)

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	colonna esistente di tipo Utf8

Campo	Tipo	Default	Vincoli
pattern	string	obbligatorio	regex valida (crate regex)
allow_null	boolean	false	se false un null è errore

Input: tabella; colonna Utf8 indicata. **Output:** input invariato; errore al primo valore non conforme al pattern.

table.coalesce

estensione · arietà: unaria · execution class: streaming · kernel v2

Config — Coalesce (quality.rs:284)

Campo	Tipo	Default	Vincoli
columns	array[string]	obbligatorio	non vuoto; colonne esistenti con tipo Arrow identico
output_column	string	obbligatorio	nome colonna valido

Input: tabella; colonne elencate in `columns` (stesso tipo Arrow). **Output:** stesse righe; colonna `output_column` aggiunta/sovrascritta con il tipo comune delle colonne (nullable); primo valore non-null per riga nell'ordine delle colonne.

Tabellari — governance

table.assert_cardinality

estensione · arietà: unaria · execution class: streaming

Config — AssertCardinality (governance.rs:14)

Campo	Tipo	Default	Vincoli
exact_rows	integer	null	se presente prevale su min_rows/max_rows
min_rows	integer	null	—
max_rows	integer	null	—

Input: tabella qualsiasi. **Output:** input invariato; errore se il numero di righe viola il vincolo (in analyze il vincolo è verificato a secco quando il row_count in input è Proven).

table.assert_metadata

estensione · arietà: unaria · execution class: streaming

Config — AssertMetadata (governance.rs:46)

Campo	Tipo	Default	Vincoli
expected	object[string → string]	obbligatorio	ogni coppia chiave/valore deve essere presente nei metadata dello schema
allow_extra	boolean	true	se false i metadata devono essere esattamente quelli attesi

Input: tabella qualsiasi (controlla i metadata dello schema Arrow). **Output:** input invariato; errore se i metadata non sono conformi.

table.assert_foreign_key

estensione · arietà: binaria (left, right) · execution class: binary-blocking

Config — ForeignKey (governance.rs:74)

Campo	Tipo	Default	Vincoli
left_keys	array[string]	obbligatorio	colonne esistenti nell'input sinistro
right_keys	array[string]	obbligatorio	colonne esistenti nell'input destro; accoppiate per posizione con left_keys, tipi Arrow identici a coppie
allow_null	boolean	false	se false una chiave null nel lato sinistro è errore

Input: due tabelle (left = referenziante, right = referenziata); chiavi con tipi Arrow identici a coppie. **Output:** input sinistro invariato; errore se una chiave di left non ha corrispondenza in right.

table.reconcile

estensione · arietà: binaria (left, right) · execution class: binary-blocking

Config — Reconcile (governance.rs:158)

Campo	Tipo	Default	Vincoli
left_keys	array[string]	obbligatorio	colonne esistenti nell'input sinistro
right_keys	array[string]	obbligatorio	colonne esistenti nell'input destro; accoppiate per posizione con left_keys, tipi Arrow identici a coppie
nulls_equal	boolean	true	se false le righe con chiave null sono contate separatamente come _only

Input: due tabelle; chiavi con tipi Arrow identici a coppie. **Output:** tabella di schema fisso `metric (Utf8) / value (UInt64)` con 5 righe: `matched_rows`, `left_only_rows`, `right_only_rows`, `left_duplicate_rows`, `right_duplicate_rows`.

Tabellari — utility

table.add_row_number

manipola-compat · arietà: unaria · execution class: blocking

Config — AddRowNumber (utility.rs:17)

Campo	Tipo	Default	Vincoli
output_column	string	"row_number"	nome non vuoto, ≤ 1024 byte
start	integer	1	—
partition_column	string	null	se presente, colonna esistente scalare/stringa; riavvia il conteggio per ogni valore distinto
order_column	string	null	deve essere null nel profilo streaming (ordinamento delegato al kernel sort)
ascending	boolean	true	attualmente ignorato dal kernel

Input: una tabella; se `partition_column` è impostata, quella colonna deve esistere. **Output:** aggiunge (o sostituisce) la colonna `output_column` di tipo `Int64` non nullable con numerazione progressiva a partire da `start`; righe e ordine invariati.

table.date_extract

manipola-compat · arietà: unaria · execution class: streaming · kernel v2

Config — DateExtract (utility.rs:95), con `DatePart` (utility.rs:81) e `InvalidDatePolicy` (utility.rs:110)

Campo	Tipo	Default	Vincoli
column	string	obbligatorio	colonna esistente scalare/stringa con valori data
parts	array[string]	["year"]	varianti: year, month, day, quarter, weekday, week, hour, minute, second
prefix	string	""	se vuoto viene usato "<column>_"
date_format	string	null	formato chrono esplicito; se omissso, parser multi-formato di default
invalid	string	"null"	varianti: null (valore non parsabile → null), error (→ errore)

Input: una tabella; la colonna `column` deve esistere (tipicamente `Utf8` con `date`). **Output:** aggiunge una colonna `Int64` nullable per ogni elemento di `parts`, denominata `<prefix><part>` (es. `data_year` con `prefix` di default); righe invariate.

table.uuid_generator

manipola-compat · arietà: unaria · execution class: streaming

Config — UuidGenerator (utility.rs:305)

Campo	Tipo	Default	Vincoli
output_column	string	"uuid"	nome non vuoto, ≤ 1024 byte

Input: una tabella qualsiasi. **Output:** aggiunge (o sostituisce) la colonna `output_column` di tipo `Utf8` non nullable con un `UUID v4` con trattini per riga; righe invariate.

Tabellari — formula

table.formula

manipola-compat · *arietà*: unaria · *execution class*: streaming

Config — Formula (formula.rs:16)

Campo	Tipo	Default	Vincoli
new_column	string	obbligatorio	nome non vuoto, ≤ 1024 byte
formula	string	obbligatorio	non vuota, ≤ 16 MiB (Limits::default().max_string_bytes); grammatica: numeri (anche notazione e/E), stringhe '.../...' (senza escape), identificatori di colonna, + - /, negazione unaria -, parentesi; ogni colonna referenziata deve esistere ed essere Int64/Float64 (numero) o scalare testuale (testo)

Input: le colonne referenziate nella formula; semantica: aritmetica tra numeri, + con almeno un operando testo = concatenazione, - * / su testo = errore, null propagato, divisione per zero = errore. Righe invariate. **Output:** colonna new_column (nullable); tipo Float64 se tutti i valori sono numeri/null (tipo inferito staticamente: Number), Utf8 se la formula produce testo. Upsert sul nome.

Tabellari — expressions

table.expression

estensione · arietà: unaria · execution class: streaming

Config — `ExpressionTransform(expressions.rs:28)`

Campo	Tipo	Default	Vincoli
output_column	string	obbligatorio	nome non vuoto, ≤ 1024 byte
expression	object	obbligatorio	AST tagged su kind (Expression, expressions.rs:37): column{name non vuoto}, literal{value scalare JSON; numero finito}, unary{op, value}, binary{op, left, right}, function{name, args ≤ 64}, case{branches 1..=64 di {when, then} (CaseBranch), else_value}. Profondità ≤ 64, nodi totali ≤ 4096. UnaryOperator: not, negate, is_null, is_not_null. BinaryOperator: add, subtract, multiply, divide, equal, not_equal, greater, greater_equal, less, less_equal, and, or. Function: coalesce (≥1 arg), null_if (2), lower/upper/trim/length/year (1), concat (≥1), contains/starts_with/ends_with (2), abs/round (1)
output_type	string	"auto"	varianti: auto, number, boolean, text (OutputType); con auto il tipo è inferito dall'AST/dai dati e tipi eterogenei tra righe sono errore

Input: le colonne referenziate dai nodi `column`; tipi ammessi: Boolean → booleano, Int64/UInt64/Float64/Decimal128/Date32/Timestamp → numero, scalare testuale → testo; altri tipi = errore. Righe invariate. **Output:** colonna `output_column` (nullable); tipo Float64 per `number`, Boolean per `boolean`, Utf8 per `text`; con `auto` il tipo è risolto sui valori (default Utf8 se tutto null). Upsert sul nome.

Geo — Geo Manipola-compat

geo.centroid

manipola-compat · arietà: unaria · execution class: streaming · CRS: projected · shape: OneToOne

Config — EmptyConfig (analyze.rs:160)

Config vuota ({}).

Input: una colonna geometria attiva (esattamente una, v1). **Output:** schema invariato; geometria trasformata in place (stesso FieldId); righe invariate.

geo.convex_hull

manipola-compat · arietà: unaria · execution class: streaming · CRS: projected · shape: OneToOne

Config — EmptyConfig (analyze.rs:160)

Config vuota ({}).

Input: una colonna geometria attiva. **Output:** schema invariato; geometria trasformata in place; righe invariate.

geo.envelope

manipola-compat · arietà: unaria · execution class: streaming · CRS: projected · shape: OneToOne

Config — EmptyConfig (analyze.rs:160)

Config vuota ({}).

Input: una colonna geometria attiva. **Output:** schema invariato; geometria trasformata in place; righe invariate.

geo.sjoin

manipola-compat · arietà: binaria (left, right) · execution class: binary-blocking · CRS: same-projected · shape: OneToMany

Config — SJoinConfig (analyze.rs:315)

Campo	Tipo	Default	Vincoli
predicate	string	obbligatorio	enum JoinPredicate (spatial_join.rs:14): intersects, contains, within, crosses, overlaps, touches

Input: due input ordinati (left, right), ciascuno con esattamente una colonna geometria attiva. **Output:** schema left + right_index (UInt64, non null); righe moltiplicate (una per coppia che soddisfa il predicato); gli attributi right si agganciano a valle con table.join.

geo.area

manipola-compat · arietà: unaria · execution class: streaming · CRS: projected · shape: OneToOne

Config — OutputColumnConfig (analyze.rs:164)

Campo	Tipo	Default	Vincoli
output_column	string	null (default "area")	non vuota; non deve collidere con colonne esistenti

Input: una colonna geometria attiva. **Output:** schema invariato + colonna Float64 (nome da output_column o area); geometria preservata; righe invariate.

geo.boundary

manipola-compat · arietà: unaria · execution class: streaming · CRS: projected · shape: OneToOne

Config — EmptyConfig (analyze.rs:160)

Config vuota ({}).

Input: una colonna geometria attiva. **Output:** schema invariato; geometria trasformata in place; righe invariate.

geo.bounds_extractor

manipola-compat · arietà: unaria · execution class: streaming · CRS: projected · shape: OneToOne

Config — EmptyConfig (analyze.rs:160)

Config vuota ({}).

Input: una colonna geometria attiva. **Output:** schema invariato + quattro colonne {geometria}_minx, {geometria}_miny, {geometria}_maxx, {geometria}_maxy (Float64, nullable); i nomi non devono collidere con colonne esistenti; righe invariate.

geo.buffer

manipola-compat · arietà: unaria · execution class: streaming · CRS: projected · shape: OneToOne

Config — `BufferConfig (analyze.rs:187)`

Campo	Tipo	Default	Vincoli
distance	number	obbligatorio	finito
cap	string	null (default round)	enum BufferCapParam (analyze.rs:171): round, flat, square

Input: una colonna geometria attiva. **Output:** schema invariato; geometria trasformata in place; righe invariate.

geo.clean_topology

manipola-compat · arietà: unaria · execution class: blocking · CRS: same-projected · shape: OneToMany

Config — `CleanTopologyConfig (analyze.rs:269)`

Campo	Tipo	Default	Vincoli
snap_tolerance	number	obbligatorio	finito, non negativo
remove_overlaps	boolean	null	—
fill_gaps	boolean	null	—

Input: una colonna geometria attiva. **Output:** schema invariato; geometria trasformata in place; righe invariate.

geo.clip

manipola-compat · arietà: binaria (left, right) · execution class: binary-blocking · CRS: same-projected · shape: OneToMany

Config — `EmptyConfig (analyze.rs:160)`

Config vuota ({}).

Input: due input ordinati (left, right), ciascuno con esattamente una colonna geometria attiva. **Output:** schema left invariato; geometria sostituita in place; righe allineate alle righe left.

geo.count_points_in_polygons

manipola-compat · arietà: binaria (left, right) · execution class: binary-blocking · CRS: same-projected · shape: OneToMany

Config — `OutputColumnConfig (analyze.rs:164)`

Campo	Tipo	Default	Vincoli
output_column	string	null (default "count")	non vuota; non deve collidere con colonne esistenti

Input: due input ordinati (left, right), ciascuno con esattamente una colonna geometria attiva. **Output:** schema left + colonna UInt64 (nome da `output_column` o `count`); righe allineate alle righe left.

geo.difference

manipola-compat · arietà: binaria (left, right) · execution class: binary-blocking · CRS: same-projected · shape: OneToMany

Config — `EmptyConfig (analyze.rs:160)`

Config vuota ({}).

Input: due input ordinati (left, right), ciascuno con esattamente una colonna geometria attiva. **Output:** schema left invariato; geometria sostituita in place; righe allineate alle righe left.

geo.dissolve

manipola-compat · arietà: unaria · execution class: blocking · CRS: projected · shape: ManyToOne

Config — `EmptyConfig (analyze.rs:160)`

Config vuota ({}).

Input: una colonna geometria attiva. **Output:** aggregazione a sole geometrie: output con la sola colonna geometria (nullable: input vuoto -> geometria null); le colonne attributo non sono propagate.

geo.distance

manipola-compat · arietà: unaria · execution class: streaming · CRS: same-projected · shape: OneToOne

Config — `OtherWkbConfig (analyze.rs:301)`

Campo	Tipo	Default	Vincoli
other_wkb	string	obbligatorio	WKB esadecimale valido (decodificabile e validato strutturalmente); CRS assunto uguale a quello dell'input
output_column	string	null (default "distance")	non vuota; non deve collidere con colonne esistenti

Input: una colonna geometria attiva; secondo operando da config (other_wkb). **Output:** schema invariato + colonna Float64 (nome da output_column o distance); geometria preservata; righe invariate.

geo.explode

manipola-compat · arietà: unaria · execution class: streaming · CRS: known · shape: OneToMany

Config — EmptyConfig (analyze.rs:160)

Config vuota ({}).

Input: una colonna geometria attiva. **Output:** schema invariato + __parent_index (UInt64, non null); espansione 1:N (piu' righe per riga di input).

geo.from_coords

manipola-compat · arietà: unaria · execution class: streaming · CRS: projected · shape: FromCoords

Config — FromCoordsConfig (analyze.rs:290)

Campo	Tipo	Default	Vincoli
x_column	string	null (default "x")	non vuota; colonna esistente di tipo Float64 o Int64
y_column	string	null (default "y")	non vuota; colonna esistente di tipo Float64 o Int64
geometry_column	string	null (default "geometry")	non vuota; non deve collidere con colonne esistenti
crs	string	null	se assente serve il CRS di piano; la definizione deve essere risolvibile

Input: nessuna colonna geometria (input non geografico); due colonne numeriche (Float64/Int64) per x e y. **Output:** schema invariato + nuova colonna geometria (WKB, nullable=false, CRS da config o di piano, dimensioni XY); l'output diventa geografico; righe invariate.

geo.intersection

manipola-compat · arietà: binaria (left, right) · execution class: binary-blocking · CRS: same-projected · shape: OneToMany

Config — EmptyConfig (analyze.rs:160)

Config vuota ({}).

Input: due input ordinati (left, right), ciascuno con esattamente una colonna geometria attiva. **Output:** schema left invariato; geometria sostituita in place; righe allineate alle righe left.

geo.length

manipola-compat · arietà: unaria · execution class: streaming · CRS: projected · shape: OneToOne

Config — OutputColumnConfig (analyze.rs:164)

Campo	Tipo	Default	Vincoli
output_column	string	null (default "length")	non vuota; non deve collidere con colonne esistenti

Input: una colonna geometria attiva. **Output:** schema invariato + colonna Float64 (nome da output_column o length); geometria preservata; righe invariate.

geo.line_builder

manipola-compat · arietà: unaria · execution class: blocking · CRS: projected · shape: ManyToOne

Config — EmptyConfig (analyze.rs:160)

Config vuota ({}).

Input: una colonna geometria attiva. **Output:** aggregazione a sole geometrie: output con la sola colonna geometria (nullable); le colonne attributo non sono propagate.

geo.nearest

manipola-compat · arietà: binaria (left, right) · execution class: binary-blocking · CRS: same-projected · shape: OneToMany

Config — NearestConfig (analyze.rs:321)

Campo	Tipo	Default	Vincoli
max_distance	number	null	se presente: finito, non negativo

Input: due input ordinati (left, right), ciascuno con esattamente una colonna geometria attiva. **Output:** schema left + __right_index (UInt64, nullable) + distance (Float64, nullable); righe left senza match entro max_distance producono null.

geo.overlay

manipola-compat · arietà: binaria (left, right) · execution class: binary-blocking · CRS: same-projected · shape: OneToMany

Config — OverlayConfig (analyze.rs:327)

Campo	Tipo	Default	Vincoli
mode	string	obbligatorio	enum OverlayMode (topology.rs:21): intersection, union, identity, symmetric_difference

Input: due input ordinati (left, right), ciascuno con esattamente una colonna geometria attiva. **Output:** sola geometria (nullable) + __left_index (UInt64, nullable) + __right_index (UInt64, nullable); le colonne attributo non sono propagate.

geo.perimeter

manipola-compat · arietà: unaria · execution class: streaming · CRS: projected · shape: OneToOne

Config — OutputColumnConfig (analyze.rs:164)

Campo	Tipo	Default	Vincoli
output_column	string	null (default "perimeter")	non vuota; non deve collidere con colonne esistenti

Input: una colonna geometria attiva. **Output:** schema invariato + colonna Float64 (nome da output_column o perimeter); geometria preservata; righe invariate.

geo.point_on_surface

manipola-compat · arietà: unaria · execution class: streaming · CRS: projected · shape: OneToOne

Config — EmptyConfig (analyze.rs:160)

Config vuota ({}).

Input: una colonna geometria attiva. **Output:** schema invariato; geometria trasformata in place; righe invariate.

geo.polygon_builder

manipola-compat · arietà: unaria · execution class: blocking · CRS: projected · shape: ManyToOne

Config — EmptyConfig (analyze.rs:160)

Config vuota ({}).

Input: una colonna geometria attiva. **Output:** aggregazione a sole geometrie: output con la sola colonna geometria (nullable); le colonne attributo non sono propagate.

geo.simplify

manipola-compat · arietà: unaria · execution class: streaming · CRS: projected · shape: OneToOne

Config — SimplifyConfig (analyze.rs:194)

Campo	Tipo	Default	Vincoli
tolerance	number	obbligatorio	finito, non negativo
policy	string	null (default douglas_peucker)	enum SimplifyPolicyParam (analyze.rs:180): douglas_peucker, preserve_topology

Input: una colonna geometria attiva. **Output:** schema invariato; geometria trasformata in place; righe invariate.

geo.symmetric_difference

manipola-compat · arietà: binaria (left, right) · execution class: binary-blocking · CRS: same-projected · shape: OneToMany

Config — EmptyConfig (analyze.rs:160)

Config vuota ({}).

Input: due input ordinati (left, right), ciascuno con esattamente una colonna geometria attiva. **Output:** schema left invariato; geometria sostituita in place; righe allineate alle righe left.

geo.to_wkt

manipola-compat · arietà: unaria · execution class: streaming · CRS: known · shape: OneToOne

Config — `OutputColumnConfig (analyze.rs:164)`

Campo	Tipo	Default	Vincoli
output_column	string	null (default "wkt")	non vuota; non deve collidere con colonne esistenti

Input: una colonna geometria attiva. **Output:** schema invariato + colonna `Utf8` (nome da `output_column` o `wkt`); geometria preservata; righe invariate.

geo.union

manipola-compat · arietà: binaria (left, right) · execution class: binary-blocking · CRS: same-projected · shape: OneToMany

Config — `EmptyConfig (analyze.rs:160)`

Config vuota (`{}`).

Input: due input ordinati (left, right), ciascuno con esattamente una colonna geometria attiva. **Output:** schema left invariato; geometria sostituita in place; righe allineate alle righe left.

geo.vertex_count

manipola-compat · arietà: unaria · execution class: streaming · CRS: known · shape: OneToOne

Config — `OutputColumnConfig (analyze.rs:164)`

Campo	Tipo	Default	Vincoli
output_column	string	null (default "vertex_count")	non vuota; non deve collidere con colonne esistenti

Input: una colonna geometria attiva. **Output:** schema invariato + colonna `UInt64` (nome da `output_column` o `vertex_count`); geometria preservata; righe invariate.

geo.voronoi

manipola-compat · arietà: unaria · execution class: blocking · CRS: projected · shape: OneToMany

Config — `VoronoiConfig (analyze.rs:277)`

Campo	Tipo	Default	Vincoli
max_points	integer	null	se presente: almeno 2

Input: una colonna geometria attiva. **Output:** schema invariato; geometria sostituita in place (celle di Voronoi); righe invariate.

geo.within

manipola-compat · arietà: binaria (left, right) · execution class: binary-blocking · CRS: same-projected · shape: OneToMany

Config — `OutputColumnConfig (analyze.rs:164)`

Campo	Tipo	Default	Vincoli
output_column	string	null (default "within")	non vuota; non deve collidere con colonne esistenti

Input: due input ordinati (left, right), ciascuno con esattamente una colonna geometria attiva. **Output:** schema left + colonna `Boolean` (nome da `output_column` o `within`); righe allineate alle righe left.

geo.make_valid

manipola-compat · arietà: unaria · execution class: streaming · CRS: known · capability: geos · shape: OneToOne · maturità: backend-pending

Config — `EmptyConfig (analyze.rs:160)`

Config vuota (`{}`).

Input: una colonna geometria attiva. **Output:** schema invariato; geometria trasformata in place; righe invariate.

geo.reproject

manipola-compat · arietà: unaria · execution class: streaming · CRS: reprojection · capability: proj · shape: OneToOne · maturità: backend-pending

Config — `ReprojectConfig (analyze.rs:201)`

Campo	Tipo	Default	Vincoli
target_crs	string	obbligatorio	definizione CRS risolvibile (riuso del CRS di piano se coincide, altrimenti backend proj)

Input: una colonna geometria attiva con CRS sorgente noto. **Output:** schema invariato; geometria riproiettata in

place; CRS del contratto e metadato `geo.crs` aggiornati a `target_crs` (unico step che modifica il CRS); righe invariate.

Geo — Predicati DE-9IM (estensioni geo)

geo.predicate_intersects

estensione · arietà: unaria · execution class: streaming · CRS: same-projected · shape: OneToOne

Config — OtherWkbConfig (analyze.rs:301)

Campo	Tipo	Default	Vincoli
other_wkb	string	obbligatorio	WKB esadecimale valido (lunghezza pari, non vuoto, decodificabile e validato strutturalmente)
output_column	string	null	non vuoto; non deve collidere con una colonna esistente; default effettivo predicate_intersects

Input: esattamente una colonna geometria (WKB); il secondo operando viene da `other_wkb` (CRS assunto uguale all'input). **Output:** schema invariato + colonna Boolean (nullable) con il nome configurato o di default; righe 1:1.

geo.predicate_disjoint

estensione · arietà: unaria · execution class: streaming · CRS: same-projected · shape: OneToOne

Config — OtherWkbConfig (analyze.rs:301)

Campo	Tipo	Default	Vincoli
other_wkb	string	obbligatorio	WKB esadecimale valido
output_column	string	null	non vuoto; non deve collidere con una colonna esistente; default effettivo predicate_disjoint

Input: esattamente una colonna geometria (WKB); secondo operando da `other_wkb`. **Output:** schema invariato + colonna Boolean (nullable) con il nome configurato o di default; righe 1:1.

geo.predicate_contains

estensione · arietà: unaria · execution class: streaming · CRS: same-projected · shape: OneToOne

Config — OtherWkbConfig (analyze.rs:301)

Campo	Tipo	Default	Vincoli
other_wkb	string	obbligatorio	WKB esadecimale valido
output_column	string	null	non vuoto; non deve collidere con una colonna esistente; default effettivo predicate_contains

Input: esattamente una colonna geometria (WKB); secondo operando da `other_wkb`. **Output:** schema invariato + colonna Boolean (nullable) con il nome configurato o di default; righe 1:1.

geo.predicate_within

estensione · arietà: unaria · execution class: streaming · CRS: same-projected · shape: OneToOne

Config — OtherWkbConfig (analyze.rs:301)

Campo	Tipo	Default	Vincoli
other_wkb	string	obbligatorio	WKB esadecimale valido
output_column	string	null	non vuoto; non deve collidere con una colonna esistente; default effettivo predicate_within

Input: esattamente una colonna geometria (WKB); secondo operando da `other_wkb`. **Output:** schema invariato + colonna Boolean (nullable) con il nome configurato o di default; righe 1:1.

geo.predicate_equals_topo

estensione · arietà: unaria · execution class: streaming · CRS: same-projected · shape: OneToOne

Config — OtherWkbConfig (analyze.rs:301)

Campo	Tipo	Default	Vincoli
other_wkb	string	obbligatorio	WKB esadecimale valido
output_column	string	null	non vuoto; non deve collidere con una colonna esistente; default effettivo predicate_equals_topo

Input: esattamente una colonna geometria (WKB); secondo operando da `other_wkb`. **Output:** schema invariato + colonna Boolean (nullable) con il nome configurato o di default; righe 1:1.

geo.predicate_covers

estensione · arietà: unaria · execution class: streaming · CRS: same-projected · shape: OneToOne

Config — OtherWkbConfig (analyze.rs:301)

Campo	Tipo	Default	Vincoli
other_wkb	string	obbligatorio	WKB esadecimale valido
output_column	string	null	non vuoto; non deve collidere con una colonna esistente; default effettivo predicate_covers

Input: esattamente una colonna geometria (WKB); secondo operando da `other_wkb`. **Output:** schema invariato + colonna Boolean (nullable) con il nome configurato o di default; righe 1:1.

geo.predicate_covered_by

estensione · arietà: unaria · execution class: streaming · CRS: same-projected · shape: OneToOne

Config — OtherWkbConfig (analyze.rs:301)

Campo	Tipo	Default	Vincoli
other_wkb	string	obbligatorio	WKB esadecimale valido
output_column	string	null	non vuoto; non deve collidere con una colonna esistente; default effettivo predicate_covered_by

Input: esattamente una colonna geometria (WKB); secondo operando da `other_wkb`. **Output:** schema invariato + colonna Boolean (nullable) con il nome configurato o di default; righe 1:1.

geo.predicate_contains_properly

estensione · arietà: unaria · execution class: streaming · CRS: same-projected · shape: OneToOne

Config — OtherWkbConfig (analyze.rs:301)

Campo	Tipo	Default	Vincoli
other_wkb	string	obbligatorio	WKB esadecimale valido
output_column	string	null	non vuoto; non deve collidere con una colonna esistente; default effettivo predicate_contains_properly

Input: esattamente una colonna geometria (WKB); secondo operando da `other_wkb`. **Output:** schema invariato + colonna Boolean (nullable) con il nome configurato o di default; righe 1:1.

geo.predicate_touches

estensione · arietà: unaria · execution class: streaming · CRS: same-projected · shape: OneToOne

Config — OtherWkbConfig (analyze.rs:301)

Campo	Tipo	Default	Vincoli
other_wkb	string	obbligatorio	WKB esadecimale valido
output_column	string	null	non vuoto; non deve collidere con una colonna esistente; default effettivo predicate_touches

Input: esattamente una colonna geometria (WKB); secondo operando da `other_wkb`. **Output:** schema invariato + colonna Boolean (nullable) con il nome configurato o di default; righe 1:1.

geo.predicate_crosses

estensione · arietà: unaria · execution class: streaming · CRS: same-projected · shape: OneToOne

Config — OtherWkbConfig (analyze.rs:301)

Campo	Tipo	Default	Vincoli
other_wkb	string	obbligatorio	WKB esadecimale valido
output_column	string	null	non vuoto; non deve collidere con una colonna esistente; default effettivo predicate_crosses

Input: esattamente una colonna geometria (WKB); secondo operando da `other_wkb`. **Output:** schema invariato + colonna Boolean (nullable) con il nome configurato o di default; righe 1:1.

geo.predicate_overlaps

estensione · arietà: unaria · execution class: streaming · CRS: same-projected · shape: OneToOne

Config — OtherWkbConfig (analyze.rs:301)

Campo	Tipo	Default	Vincoli
other_wkb	string	obbligatorio	WKB esadecimale valido
output_column	string	null	non vuoto; non deve collidere con una colonna esistente; default effettivo predicate_overlaps

Input: esattamente una colonna geometria (WKB); secondo operando da `other_wkb`. **Output:** schema invariato + colonna Boolean (nullable) con il nome configurato o di default; righe 1:1.

Geo — Estensioni geo

geo.affine_transform

estensione · arietà: unaria · execution class: streaming · CRS: projected · shape: OneToOne

Config — `AffineTransformConfig (analyze.rs:207)`

Campo	Tipo	Default	Vincoli
coefficients	array[number]	obbligatorio	esattamente 6 elementi; tutti finiti

Input: esattamente una colonna geometria (WKB). **Output:** schema e righe invariati; geometria trasformata in place (stesso FieldId).

geo.translate

estensione · arietà: unaria · execution class: streaming · CRS: projected · shape: OneToOne

Config — `TranslateConfig (analyze.rs:213)`

Campo	Tipo	Default	Vincoli
x_offset	number	obbligatorio	finito
y_offset	number	obbligatorio	finito

Input: esattamente una colonna geometria (WKB). **Output:** schema e righe invariati; geometria trasformata in place (stesso FieldId).

geo.scale

estensione · arietà: unaria · execution class: streaming · CRS: projected · shape: OneToOne

Config — `ScaleConfig (analyze.rs:220)`

Campo	Tipo	Default	Vincoli
x_factor	number	obbligatorio	finito
y_factor	number	obbligatorio	finito
x_origin	number	null	finito se presente
y_origin	number	null	finito se presente

Input: esattamente una colonna geometria (WKB). **Output:** schema e righe invariati; geometria trasformata in place (stesso FieldId).

geo.rotate

estensione · arietà: unaria · execution class: streaming · CRS: projected · shape: OneToOne

Config — `RotateConfig (analyze.rs:229)`

Campo	Tipo	Default	Vincoli
degrees	number	obbligatorio	finito
x_origin	number	null	finito se presente
y_origin	number	null	finito se presente

Input: esattamente una colonna geometria (WKB). **Output:** schema e righe invariati; geometria trasformata in place (stesso FieldId).

geo.concave_hull

estensione · arietà: unaria · execution class: streaming · CRS: projected · shape: OneToOne

Config — `ConcaveHullConfig (analyze.rs:237)`

Campo	Tipo	Default	Vincoli
concavity	number	obbligatorio	> 0, finito
length_threshold	number	null	>= 0, finito se presente

Input: esattamente una colonna geometria (WKB). **Output:** schema e righe invariati; geometria sostituita in place (stesso FieldId).

geo.hausdorff_distance

estensione · arietà: unaria · execution class: streaming · CRS: same-projected · shape: OneToOne

Config — `OtherWkbConfig (analyze.rs:301)`

Campo	Tipo	Default	Vincoli
other_wkb	string	obbligatorio	WKB esadecimale valido
output_column	string	null	non vuoto; non deve collidere con una colonna esistente; default effettivo hausdorff_distance

Input: esattamente una colonna geometria (WKB); secondo operando da `other_wkb`. **Output:** schema invariato + colonna Float64 (nullable) con il nome configurato o di default; righe 1:1.

geo.haversine_distance

estensione · arietà: unaria · execution class: streaming · CRS: geographic · shape: OneToOne

Config — OtherWkbConfig (analyze.rs:301)

Campo	Tipo	Default	Vincoli
other_wkb	string	obbligatorio	WKB esadecimale valido
output_column	string	null	non vuoto; non deve collidere con una colonna esistente; default effettivo haversine_distance

Input: esattamente una colonna geometria (WKB); secondo operando da `other_wkb`. **Output:** schema invariato + colonna Float64 (nullable) con il nome configurato o di default; righe 1:1.

geo.geodesic_distance

estensione · arietà: unaria · execution class: streaming · CRS: geographic · shape: OneToOne

Config — OtherWkbConfig (analyze.rs:301)

Campo	Tipo	Default	Vincoli
other_wkb	string	obbligatorio	WKB esadecimale valido
output_column	string	null	non vuoto; non deve collidere con una colonna esistente; default effettivo geodesic_distance

Input: esattamente una colonna geometria (WKB); secondo operando da `other_wkb`. **Output:** schema invariato + colonna Float64 (nullable) con il nome configurato o di default; righe 1:1.

geo.geodesic_line_length

estensione · arietà: unaria · execution class: streaming · CRS: geographic · shape: OneToOne

Config — OutputColumnConfig (analyze.rs:164)

Campo	Tipo	Default	Vincoli
output_column	string	null	non vuoto; non deve collidere con una colonna esistente; default effettivo geodesic_line_length

Input: esattamente una colonna geometria (WKB). **Output:** schema invariato + colonna Float64 (nullable) con il nome configurato o di default; righe 1:1.

geo.densify

estensione · arietà: unaria · execution class: streaming · CRS: projected · shape: OneToOne

Config — DensifyConfig (analyze.rs:244)

Campo	Tipo	Default	Vincoli
max_segment_length	number	obbligatorio	> 0, finito

Input: esattamente una colonna geometria (WKB). **Output:** schema e righe invariati; geometria sostituita in place (stesso FieldId).

geo.snap_to_grid

estensione · arietà: unaria · execution class: streaming · CRS: projected · shape: OneToOne

Config — SnapToGridConfig (analyze.rs:250)

Campo	Tipo	Default	Vincoli
grid_size	number	obbligatorio	> 0, finito

Input: esattamente una colonna geometria (WKB). **Output:** schema e righe invariati; geometria sostituita in place (stesso FieldId).

geo.delaunay

estensione · arietà: unaria · execution class: blocking · CRS: projected · shape: OneToMany

Config — Config vuota ({}).

Input: esattamente una colonna geometria (WKB). **Output:** schema invariato + colonna `__parent_index` (UInt64, non null); espansione 1:N (più righe per riga di input).

geo.polygonize

estensione · arietà: unaria · execution class: blocking · CRS: projected · capability: geos · shape: ManyToOne · maturità: backend-pending

Config — PolygonizeConfig (analyze.rs:283)

Campo	Tipo	Default	Vincoli
node_input	boolean	null	—
require_complete	boolean	null	—

Input: esattamente una colonna geometria (WKB). **Output:** aggregazione a sole geometrie: le colonne attributo non sono propagate; output = colonna geometria (nullable) + colonna `__class` (Utf8, non null).

geo.line_merge

estensione · arietà: unaria · execution class: blocking · CRS: projected · shape: ManyToOne

Config — Config vuota ({}).

Input: esattamente una colonna geometria (WKB). **Output:** aggregazione a sole geometrie: le colonne attributo non sono propagate; output = sola colonna geometria (nullable).

geo.split

estensione · arietà: unaria · execution class: streaming · CRS: same-projected · capability: geos · shape: OneToMany · maturità: backend-pending

Config — SplitConfig (analyze.rs:308)

Campo	Tipo	Default	Vincoli
other_wkb	string	obbligatorio	WKB esadecimale valido
tolerance	number	null	>= 0, finito se presente

Input: esattamente una colonna geometria (WKB); secondo operando da `other_wkb`. **Output:** schema invariato + colonna `__parent_index` (UInt64, non null); espansione 1:N (più righe per riga di input).

geo.line_substring

estensione · arietà: unaria · execution class: streaming · CRS: projected · shape: OneToOne

Config — LineSubstringConfig (analyze.rs:256)

Campo	Tipo	Default	Vincoli
start_ratio	number	obbligatorio	finito, in [0, 1]
end_ratio	number	obbligatorio	finito, in [0, 1]; start_ratio <= end_ratio

Input: esattamente una colonna geometria (WKB). **Output:** schema e righe invariati; geometria sostituita in place (stesso FieldId).

geo.line_interpolate_point

estensione · arietà: unaria · execution class: streaming · CRS: projected · shape: OneToOne

Config — LineInterpolatePointConfig (analyze.rs:263)

Campo	Tipo	Default	Vincoli
ratio	number	obbligatorio	finito, in [0, 1]

Input: esattamente una colonna geometria (WKB). **Output:** schema e righe invariati; geometria sostituita in place (stesso FieldId).

geo.frechet_distance

estensione · arietà: unaria · execution class: streaming · CRS: same-projected · shape: OneToOne

Config — OtherWkbConfig (analyze.rs:301)

Campo	Tipo	Default	Vincoli
other_wkb	string	obbligatorio	WKB esadecimale valido
output_column	string	null	non vuoto; non deve collidere con una colonna esistente; default effettivo <code>frechet_distance</code>

Input: esattamente una colonna geometria (WKB); secondo operando da `other_wkb`. **Output:** schema invariato + colonna Float64 (nullable) con il nome configurato o di default; righe 1:1.

geo.bearing

estensione · arietà: unaria · execution class: streaming · CRS: geographic · shape: OneToOne

Config — `OtherWkbConfig (analyze.rs:301)`

Campo	Tipo	Default	Vincoli
<code>other_wkb</code>	string	obbligatorio	WKB esadecimale valido
<code>output_column</code>	string	null	non vuoto; non deve collidere con una colonna esistente; default effettivo bearing

Input: esattamente una colonna geometria (WKB); secondo operando da `other_wkb`. **Output:** schema invariato + colonna Float64 (nullable) con il nome configurato o di default; righe 1:1.

geo.geodesic_area

estensione · arietà: unaria · execution class: streaming · CRS: geographic · shape: OneToOne

Config — `OutputColumnConfig (analyze.rs:164)`

Campo	Tipo	Default	Vincoli
<code>output_column</code>	string	null	non vuoto; non deve collidere con una colonna esistente; default effettivo geodesic_area

Input: esattamente una colonna geometria (WKB). **Output:** schema invariato + colonna Float64 (nullable) con il nome configurato o di default; righe 1:1.

geo.geometry_diagnostics

estensione · arietà: unaria · execution class: streaming · CRS: known · shape: Diagnostic

Config — `Config vuota ({}).`

Input: esattamente una colonna geometria (WKB). **Output:** la colonna geometria è **sostituita** (nella stessa posizione) dalle 10 colonne diagnostiche, tutte nullable: `geometry_type (Utf8)`, `coordinate_count (UInt64)`, `is_empty (Boolean)`, `is_finite (Boolean)`, `is_valid (Boolean)`, `validity_reason (Utf8)`, `bounds_minx (Float64)`, `bounds_miny (Float64)`, `bounds_maxx (Float64)`, `bounds_maxy (Float64)`; il contratto diventa non-geografico; righe 1:1.